Graphs with Python

Mapping Complex Relationships & Non-Linear Logic

Series 07: Discrete Mathematical Structures

Prepared for: The CodeHive Developer Portal

Document Outline

01. Understanding Graphs

A **Graph** is a non-linear data structure consisting of two fundamental components: **Vertices** (also known as nodes) and **Edges** (the connections between them).

> **Non-Linearity:** Unlike Arrays or Linked Lists, which describe a strict sequence, a Graph allows for multiple paths between any two points. It represents a web of connections rather than a chain.

The Building Blocks

- **Vertices (V):** The objects in the graph (e.g., users, cities, web pages).
- **Edges (—):** The relationships. They can be undirected (friendship) or directed (social media followers).

Graphs allow us to model systems where the relationship is just as important as the data itself.

02. Real-World Applications

Graphs are the backbone of modern technology. Without them, most of our daily digital experiences would be impossible.

1. Social Networks

Platforms like Facebook or LinkedIn model users as vertices. An edge exists if they are friends. Graph algorithms suggest "People you may know" by analyzing mutual connections.

2. Navigation & Maps

Locations are vertices and roads are edges. GPS applications use **Dijkstra's Algorithm** or **A* Search** on these graphs to find the shortest path between destinations.

3. Search Engines (The Internet)

The World Wide Web is a massive directed graph where pages are vertices and hyperlinks are directed edges. Google's PageRank algorithm assigns value to pages based on how many edges point to them.

4. Biology & Neural Networks

Graphs model protein interactions and neural pathways in the human brain, allowing researchers to visualize complex biological systems.

03. Representing Graphs: The Matrix

An **Adjacency Matrix** is a 2D array (matrix) where the cell at index `(i, j)` tells us if there is an edge between vertex `i` and vertex `j`.

Undirected Matrix Representation

In an undirected graph, if A is connected to B, B is also connected to A. This creates a **symmetrical matrix**.

Vertex	A	B	C	D
A	0	1	1	1
B	1	0	0	1
C	1	0	0	0
D	1	1	0	0

Memory Usage: This method requires V^2 space. While fast for checking if two nodes are connected, it can be wasteful for graphs with many nodes but few connections.

04. Weighted and Directed Variations

Basic matrices use 1 for an edge and `0` for none. For complex maps, we need to express "how far" (weight) and "what direction."

Directed and Weighted Matrix

If there is an edge from A to B with a cost of 3, we store the value 3 at `Matrix[A][B]`. If there is no return path, `Matrix[B][A]` remains `0` or `None`.

From \ To	A	B	C	D
A	-	3 (w)	-	-
B	-	-	2 (w)	-
C	-	-	-	-
D	4 (w)	-	1 (w)	-

This allows for asymmetric relationships, such as one-way streets or one-sided social follows.

05. Graph Efficiency: Adjacency Lists

When a graph is **sparse** (few edges), an Adjacency List is far more efficient than a matrix. Instead of a grid, each vertex maintains a list of only its neighbors.

The Structure

1. An array containing the vertices.
2. Each array index points to a **Linked List** or **Dynamic Array** of neighboring vertex indices.

Example for Vertex A:
Vertex A → [B, C, D]
Vertex B → [A, D]
Vertex C → [A]

This saves significant memory because we do not reserve space for edges that don't exist.

06. Sparse vs. Dense Graphs

The choice between Matrix and List depends on the **density** of the graph—the ratio of edges to vertices.

Criterion	Adjacency Matrix	Adjacency List
Best for...	Dense Graphs (many edges)	Sparse Graphs (few edges)
Checking (A, B) Edge	O(1) - Instant	O(V) - Must search list
Finding Neighbors	O(V) - Scan entire row	O(Degree) - Direct access
Space Used	V^2 (Constant)	$V + E$ (Scales with edges)

07. Adjacency List Complexity

Adjacency Lists can also easily handle directed and weighted graphs by storing objects or pairs in the list.

Weighted Representation

Instead of just storing the neighbor's index, the list stores a pair: `(neighbor_index, weight)`.

Vertex D: [(A, 4), (C, 1)]

(Meaning: Edge to A with weight 4, Edge to C with weight 1)

Why use Lists in Python?

Python's `dictionary` and `list` objects make implementing this structure trivial. You can represent a graph as a dictionary where keys are nodes and values are lists of neighbor tuples.

08. Strategic Summary

Graphs are the ultimate structure for modeling the modern world. Their versatility across social, navigational, and technical domains makes them a core requirement for any software engineer.

Key Takeaways

- **Matrix:** Best when we need instant edge-checks ($O(1)$) and space is not an issue.
- **List:** Best for massive but sparse systems like social networks or street maps.
- **Directed:** Essential for relationships where "A follows B" doesn't mean "B follows A."
- **Weighted:** Essential for representing the cost of a path (distance, time, bandwidth).

CODEHIVE ACADEMY